

Semaphores

Counting saved wakeups with atomic operations

Definition

A **semaphore** is an integer synchronization variable used to count saved wakeups and control access to shared resources.

1. Why Semaphores Were Introduced

Main Idea

The sleep-wakeup solution failed because a wakeup could be sent before the target process was actually asleep. That wakeup was lost.

Dijkstra's Idea

E. W. Dijkstra proposed storing wakeups in an integer variable. A semaphore value of 0 means no wakeups are saved. A positive value means one or more wakeups are available for future use.

Problem Being Solved

The goal is to avoid lost wakeups while still blocking processes instead of forcing them to busy-wait.

2. Semaphore Operations

Two Basic Operations

Semaphores use two atomic operations:

- **down**: uses one saved wakeup or blocks if none is available.
- **up**: creates one saved wakeup or wakes one blocked process.

Operation	If Semaphore > 0	If Semaphore $= 0$
down	Decrement the semaphore and continue.	Block the process before completing the operation.
up	Increment the semaphore.	If processes are sleeping on it, wake one of them.

Atomicity Requirement

Checking the semaphore value, changing it, and possibly putting a process to sleep must happen as one indivisible atomic action.

3. The Down Operation

How down Works

down checks whether the semaphore value is greater than 0. If it is, the value is decremented and the process continues. If the value is 0, the process is put to sleep.

```
down(semaphore s)
{
    if (s > 0) {
        s = s - 1;
        continue;
    } else {
        sleep on s;
    }
}
```

Important

The code above is only conceptual. A real **down** operation must be implemented atomically by the operating system.

4. The Up Operation

How up Works

up increments the semaphore. If one or more processes are blocked on that semaphore, the system chooses one of them and allows it to complete its earlier **down**.

```
up(semaphore s)
{
    if (processes are sleeping on s) {
        wake one process blocked on s;
    } else {
        s = s + 1;
    }
}
```

No Blocking on Up

A process never blocks while doing **up**. The operation either saves a wakeup by increasing the semaphore or wakes one process.

5. Why Atomicity Matters

Atomic Action

An **atomic action** is a group of related operations that is performed without interruption. Other processes cannot observe or interfere with the middle of the operation.

Semaphore Safety

Once a semaphore operation starts, no other process may access that semaphore until the operation has completed or the calling process has blocked.

Without Atomicity

If **down** could be interrupted after checking the semaphore but before sleeping, semaphores would have the same lost-wakeup problem as ordinary **sleep** and **wakeup**.

6. Old Names: P and V

Historical Note

Dijkstra originally called the operations **P** and **V**. Modern texts often use **down** and **up** because those names describe the effect on the semaphore value more clearly.

7. Producer-Consumer Using Semaphores

Goal

Use semaphores to solve the producer-consumer problem without lost wakeups and without busy waiting.

The solution uses three semaphores:

Semaphore	Initial Value	Purpose
mutex	1	Controls access to the critical region that modifies the buffer.
empty	N	Counts empty buffer slots.
full	0	Counts full buffer slots.

```
#define N 100 /* number of slots in the buffer */
typedef int semaphore; /* semaphores are a special kind of int */

semaphore mutex = 1; /* controls access to critical region */
semaphore empty = N; /* counts empty buffer slots */
semaphore full = 0; /* counts full buffer slots */

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}
```

```

}

void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}

```

8. How the Producer Works

Step	Meaning
<code>produce_item()</code>	Create an item outside the critical region.
<code>down(&empty)</code>	Wait until at least one empty buffer slot exists.
<code>down(&mutex)</code>	Enter the critical region safely.
<code>insert_item(item)</code>	Put the item into the shared buffer.
<code>up(&mutex)</code>	Leave the critical region.
<code>up(&full)</code>	Announce that one more full slot exists.

9. How the Consumer Works

Step	Meaning
<code>down(&full)</code>	Wait until at least one full buffer slot exists.
<code>down(&mutex)</code>	Enter the critical region safely.
<code>remove_item()</code>	Remove one item from the shared buffer.
<code>up(&mutex)</code>	Leave the critical region.
<code>up(&empty)</code>	Announce that one more empty slot exists.
<code>consume_item(item)</code>	Use the item outside the critical region.

10. Why This Solves the Lost-Wakeup Problem

Stored Wakeups

The semaphores `empty` and `full` store information about available slots. If an `up` happens before another process does `down`, the semaphore value records that event.

Example

If the producer inserts an item and does `up(&full)` before the consumer waits, `full` becomes positive. Later, the consumer's `down(&full)` consumes that saved wakeup instead of sleeping.

Order Matters

The producer must do `down(&empty)` before entering the buffer, and the consumer must do `down(&full)` before removing from the buffer. Otherwise, the producer could overfill the buffer or the consumer could remove from an empty buffer.

11. Binary and Counting Semaphores

Binary Semaphore

A **binary semaphore** is initialized to 1 and is used to ensure that only one process at a time enters a critical region.

Counting Semaphore

A **counting semaphore** can hold values larger than 1 and is used to count available resources or saved events.

Semaphore	Type	Use in Producer-Consumer
<code>mutex</code>	Binary	Guarantees mutual exclusion for buffer access.
<code>empty</code>	Counting	Counts empty buffer slots.
<code>full</code>	Counting	Counts full buffer slots.

12. Implementation of Semaphores

Single CPU

The operating system can implement `up` and `down` as system calls and briefly disable interrupts while testing and updating the semaphore.

Multiple CPUs

On multiprocessor systems, each semaphore must be protected so that only one CPU examines or updates it at a time. Hardware instructions such as TSL or XCHG can be used for this short internal lock.

Important Distinction

Using TSL or XCHG briefly inside the semaphore implementation is different from making the producer or consumer busy-wait for the buffer. The internal semaphore operation takes only a few instructions; waiting for a buffer slot may take much longer.

13. Semaphores and I/O Interrupts

Device Manager Pattern

An operating system can associate a semaphore, initially 0, with each I/O device. After starting an I/O operation, the device manager does **down** on that semaphore and blocks.

When the device interrupt arrives, the interrupt handler does **up** on the device's semaphore. This makes the device manager ready to run again.



14. Two Uses of Semaphores

Use	Semaphore Example	Purpose
Mutual exclusion	<code>mutex</code>	Ensures only one process at a time reads or writes the shared buffer.
Synchronization	<code>full</code> , <code>empty</code>	Ensures events happen in a valid order: do not consume from an empty buffer or produce into a full one.

Core Distinction

Mutual exclusion protects a critical region. Synchronization controls the order in which processes are allowed to proceed.

15. Quick Review

Key Points

- A semaphore is an integer used to count saved wakeups.
- **down** decrements the semaphore or blocks if the value is 0.
- **up** increments the semaphore or wakes one blocked process.
- Semaphore operations must be atomic.
- Binary semaphores are used for mutual exclusion.
- Counting semaphores are used to count resources or saved events.
- In producer-consumer, `mutex` protects the buffer.
- `empty` counts empty slots, and `full` counts full slots.
- Semaphores can also be used to block device managers until I/O interrupts occur.